UNIVERSIDADE FEDERAL DO RIO GRANDE DO SUL
INSTITUTO DE INFORMÁTICA
DEPARTAMENTO DE INFORMÁTICA APLICADA
INF01120 – DESENVOLVIMENTO DE SOFTWARE
PROFESSOR: Marcelo Soares Pimenta (*mpimenta@inf.ufrgs.br*)

Enunciado do Trabalho Prático – Fase 3

Introdução

LEIAM ATÉ O FIM este enunciado !!!!!

Atividade de Revisão por Pares: Identificação de Bad Smells [e Sugestões de Refatoração]

1. Objetivos desta atividade

- Aplicar na prática os conceitos de **bad smells** no código [e refatoração], estudados em aula.
- Desenvolver a capacidade de **análise crítica** de código alheio, identificando pontos de melhoria à luz dos princípios SOLID e boas práticas de programação.
- Simular um cenário real de revisão de código (*code review*), elaborada em grupo.
- Fornecer **feedback construtivo** para que outro grupo possa evoluir a qualidade do seu projeto.

2. Descrição da Atividade

Cada grupo terá acesso ao código fonte (repositório Git) de **outro grupo** da disciplina, indicado pelo professor. O grupo autor do trabalho a ser revisado será doravante denominado GRUPO AUTOR. O grupo que realizará a revisão será doravante denominado GRUPO REVISOR.

O grupo revisor deverá:

- **Analisar o código** do projeto "Gerador de Música a partir de Texto" produzido pelo grupo autor designado.
- Utilizar para esta avaliação **ao menos 6 *bad smells*** (do checklist de 10 *bad smells* fornecido – ver final deste enunciado) para aplicar na revisão. A escolha deve ser justificada pela relevância destes *bad smells* para o código analisado.
- Além disso, o grupo deverá **propor 4 novos critérios de avaliação** que **não estão entre os 10 presentes no checklist original** de *bad smells* fornecido. Esses novos itens devem estar alinhados com os conceitos da disciplina (ex: princípios SOLID, modularidade, legibilidade, testabilidade, usabilidade, etc.) e devem ser claramente definidos e justificados.
- Para cada critério identificado (tanto nos 6 bad smells escolhidos quanto nos 4 propostos), **documentar**:
  - O local exato (classe, método ou linha) onde o problema foi encontrado.
  - Uma breve descrição do problema e por que ele é relevante.

- Opcionalmente, o grupo revisor pode fazer uma **sugestão de refatoração** concreta, explicando como o código poderia ser melhorado (pode incluir trechos de código ilustrativos).

Ao final, produzir um **relatório de revisão** contendo:

- **Introdução:** Explicação dos critérios utilizados (quais 6 smells da lista original foram escolhidos e por quê; quais 4 novos critérios foram criados para a revisão e as justificativas para cada um deles).
- **Análise do código:** Para cada critério, liste os problemas encontrados (se houver) com localização, descrição e, se for o caso, sugestão de refatoração.
- **Pontos fortes observados:** Destaque aspectos positivos do código que merecem ser mantidos.
- **Conclusão:** Síntese das principais recomendações.

3. Passos a serem realizados:

1. **Acesso ao código:** Cada grupo autor deverá disponibilizar o link do repositório Git do ao grupo revisor. O grupo revisor deverá clonar o repositório (para evitar problemas de integridade) e explorar o código.

2. **Definição dos critérios:** Antes de iniciar a análise, o grupo revisor deve apresentar quais 6 *smells* estudados foram considerados mais adequados para avaliar o projeto e por quê, e quais 4 novos critérios o grupo deseja propor e por quê. Registre essas decisões e suas justificativas.

3. **Análise sistemática:** O grupo revisor deve percorrer o código procurando evidências de problemas de acordo com os 10 critérios definidos (6+4). Utilize ferramentas de análise estática ou ferramentas de IA se desejar, mas a avaliação principal deve ser VERIFICADA PELO GRUPO REVISOR, que é responsável pelo resultado da revisão.

4. **Documentação:** O grupo revisor deve criar um relatório de revisão em formato PDF, seguindo a estrutura sugerida. Se possível, inclua referências aos trechos para ilustrar.

5. **Entrega:** O relatório deverá ser postado no Moodle (UMA POSTAGEM por grupo revisor) até a data estipulada. **O grupo revisor DEVE enviar ao grupo autor uma cópia do relatório para que possa refletir sobre o que foi apontado na revisão e fazer as correções possíveis.**

5. Quais ferramentas usar para Análise Sistemática?

Para realizar uma **correção automatizada baseada em princípios** de design (SOLID, acoplamento, coesão), a melhor estratégia é usar **ferramentas** que extraiam **métricas** de

software. Como você precisa de **opções gratuitas ou open source**, aqui estão as mais recomendadas divididas por linguagem e propósito:

a. Java: CK (A mais completa para SOLID)

Se o foco é acadêmico ou puramente técnico sobre design de código, a biblioteca CK é a melhor escolha. Ela foca especificamente em métricas de classes.

O que mede: CBO (Acoplamento entre objetos), LCOM (Falta de coesão em métodos - ideal para SRP), DIT (Profundidade de herança) e RFC (Resposta por classe).

- Licença: Open Source (GitHub). Ver: mauricioaniche/ck no GitHub.

b. Multi-linguagem: SonarQube (Community Edition)

O SonarQube é o padrão de mercado para qualidade de código. A versão Community é gratuita para auto-hospedagem e analisa mais de 30 linguagens.

O que mede: Débito técnico, complexidade ciclomática, duplicação de código e "Code Smells" que violam boas práticas.

Destaque: Possui regras prontas que detectam violações de design, como métodos muito longos ou classes com muitas responsabilidades.

- Ver: SonarQube Community Edition.

c. Python: Radon e Pylint

Para Python, a combinação dessas duas ferramentas cobre quase tudo:

Radon: Foca em métricas puras como Complexidade Ciclomática e Índice de Manutenibilidade. É leve e fácil de integrar em scripts de correção automática.

Pylint: Verifica se o código segue o PEP8 e possui verificações de design (ex: "too many instance attributes"), o que indica baixa coesão.

- Ver: Radon Documentation e Pylint Project.

d. JavaScript/TypeScript: Codehawk e ESLint

Codehawk: Uma ferramenta específica para medir a complexidade e a "saúde" de projetos JS/TS, ajudando a identificar áreas com alto acoplamento.

ESLint: Embora seja um linter, possui plugins como o eslint-plugin-import que ajudam a controlar o acoplamento entre módulos.

- Ver: Codehawk no GitHub.

Resumo de Recomendação

Necessidade	Ferramenta Recomendada	Por que usar?
Análise Profunda (SOLID)	CK (Java)	Focada em métricas clássicas de OO (LCOM, CBO).
Plataforma Completa	SonarQube	Melhor interface e histórico de evolução do aluno.
Scripts Rápidos	Radon (Python)	Extremamente simples de rodar via terminal ou API.
Segurança e Qualidade	Snyk ou Semgrep	Opções leves (para CI/CD) que pegam erros de lógica.

5. O que Deve Ser Entregue pelo Grupo Revisor

O grupo revisor deve entregar:

- Um arquivo contendo o **relatório de revisão** (formato PDF)
- Opcionalmente, o grupo pode anexar um **arquivo de anotações** ou checklist preenchido com as marcações.

6. Critérios de Avaliação (para o grupo revisor)

A atividade será avaliada com base nos seguintes aspectos:

Critério	Peso
Justificativa dos critérios escolhidos: Clareza e pertinência na escolha dos 6 smells da lista original e na criação dos 4 novos critérios.	20%
Identificação correta de problemas: Os pontos apontados são realmente problemas de código (e não apenas opiniões subjetivas sem fundamento).	50%
Qualidade das sugestões de refatoração: As propostas são práticas, viáveis e alinhadas com boas práticas (SOLID, modularidade, etc.). Se não houver sugestões, esta nota terá valor Zero.	10%
Clareza e organização do relatório: O documento é bem estruturado, fácil de ler, e contém exemplos concretos.	10%

Critério	Peso
Profundidade da análise: A revisão demonstra compreensão do código e vai além da superfície, explorando aspectos de design.	10%

Observação: O grupo autor (que recebeu a revisão) **não** será avaliado pelo código em si nesta atividade; o foco é a capacidade de revisão do grupo revisor. No entanto, o feedback recebido deverá ser usado pelo grupo autor para melhorar o projeto final.

7. O que deve ser feito pelo Grupo Autor após a revisão

O grupo autor deve ler minuciosamente o **relatório de revisão** entregue pelo grupo revisor. **A partir das críticas, sugestões e recomendações feitas pelo grupo revisor, o grupo autor deve alterar o seu software elaborado na Fase 2 do Trabalho Prático para levá-las em consideração tanto quanto possível, para a apresentação e demonstração ao professor no final da Fase 3.**

Durante a apresentação oral do trabalho do grupo autor , **o grupo deve colocar em slides um RESUMO das observações feitas pelo grupo revisor e apontar o que foi feito no software em resposta a cada uma delas.**

8. Datas de Entrega

- **Disponibilização do Acesso ao Código aos revisores:** [data a ser definida em sala e postada no moodle]
- **Entrega do relatório de revisão:** [data a ser definida em sala e postada no moodle]

9. Instruções Finais

O grupo revisor deve ser **respeitoso e construtivo** nos comentários. O objetivo é ajudar o outro grupo (grupo autor) a crescer, e não criticar negativamente.
Lembre-se de que os 4 novos critérios devem ser **relevantes** para o contexto do projeto. Eles podem abordar aspectos como: aderência aos princípios SOLID (além dos smells), uso adequado de herança/composição, tratamento de exceções, documentação do código, coesão de módulos, etc.
Dúvidas sobre a atividade devem ser esclarecidas em **aulas de acompanhamento** OU encaminhadas ao professor via Moodle ou e-mail. **Aulas de acompanhamento** serão agendadas para resolução de dúvidas ou discussão do trabalho com os colegas e o professor. Uma vez decididas, as datas serão divulgadas e postadas no moodle da disciplina.

Aproveite esta oportunidade para exercitar o olhar crítico e aprender com as soluções (e problemas) dos colegas.

Veja a seguir (ver abaixo) um CHECKLIST de *bad smells* para ser usado neste trabalho (Fase 3).

Bom trabalho a todos!

= = = =

UNIVERSIDADE FEDERAL DO RIO GRANDE DO SUL
INSTITUTO DE INFORMÁTICA
DEPARTAMENTO DE INFORMÁTICA APLICADA
INF01120 – Desenvolvimento de Software
PROFESSOR: Marcelo Soares Pimenta (*mpimenta@inf.ufrgs.br*)

CHECKLIST de Bad Smells
(para usar no Trabalho Prático – FASE 3)

Introdução

Com base nos resultados da busca que tratam dos "code smells" clássicos da literatura de engenharia de software , preparei um checklist focado nos 10 maus-cheiros mais significativos. O objetivo é que cada grupo utilize esta lista para avaliar o código do projeto **Gerador de Música a partir de Texto**, especialmente considerando as mudanças de requisitos que vieram.

A tabela abaixo descreve cada smell, como identificá-lo no contexto do trabalho e como corrigi-lo, promovendo a aplicação dos princípios SOLID e boas práticas de programação.

☑ Checklist de Verificação de Código (Code Smells)

Use este checklist durante as revisões de código do grupo. Para cada item, marque **OK** se o código está livre do smell, ou **Refatorar** se o smell for identificado e precisar de correção.

Smell (Mau-Cheiro)	Onde Observar no Projeto (Sinais e Sintomas)	Como Corrigir (Refatorações Sugeridas)
1. Código Duplicado O mais fedorento	• A mesma lógica de mapeamento (ex: `if` para 'A', `if` para 'B') repetida em vários lugares.	• **Extrair Método (Extract Method):** Crie um método privado para a lógica duplicada.

Smell (Mau-Cheiro)	Onde Observar no Projeto (Sinais e Sintomas)	Como Corrigir (Refatorações Sugeridas)
de todos.	• Algoritmos de interpretação de texto idênticos em classes diferentes. • Estruturas `switch` ou cadeias de `if-else` que testam as mesmas condições repetidamente.	• **Extrair Classe (Extract Class):** Se a duplicação envolve dados e comportamento, crie uma nova classe. • **Puxar Método para Superclasse (Pull Up Method):** Se a duplicação está em subclasses.
2. Classe Grande (Large Class) Uma classe que tenta fazer tudo sozinha.	• Uma classe `GeradorMusical` que contém regras de mapeamento, manipulação de contexto *e* geração de áudio. • Classes com dezenas de métodos ou centenas de linhas. • Dificuldade em descrever a responsabilidade única da classe.	• **Extrair Classe (Extract Class):** Separe responsabilidades distintas em novas classes (ex: `RuleEngine`, `MusicContext`, `MidiGenerator`). • **Extrair Superclasse (Extract Superclass):** Crie uma classe base para comportamentos comuns.
3. Método Longo (Long Method) Métodos que fazem tudo e são difíceis de ler.	• Um método `processarTexto()` com mais de 20-30 linhas, cheio de loops e condicionais aninhados. • Métodos que exigem rolagem para serem lidos por completo. • Comentários explicando blocos de código (muitas vezes indicam que o bloco deveria ser um método).	• **Extrair Método (Extract Method):** Quebre o mé menores e bem nomeados (ex: `processarNota()`, `processarMudancaInstru lume()`).
4. Lista de Parâmetros Longa (Long Parameter List) Métodos que recebem muitos argumentos.	• Um método `gerarNota(String tom, int oitava, int volume, int instrumento, long duracao, boolean isPausa)`. • Dificuldade em lembrar a ordem e o significado dos parâmetros.	• **Introduzir Objeto-Parâmetro (Introduce Parameter Object):** Agrupe parâmetros relacionados em um objeto (ex: uma classe `Nota` que contém tom, oitava, volume). • **Substituir Parâmetro por Método (Replace Parameter with Query):** Se o valor pode ser obtido de outro objeto.

Smell (Mau-Cheiro)	Onde Observar no Projeto (Sinais e Sintomas)	Como Corrigir (Refatorações Sugeridas)
5. Inveja de Recursos (Feature Envy) Um método que parece mais interessado em outra classe do que na sua própria.	• Na classe `RuleEngine`, um método que acessa constantemente os dados de `MusicContext` para *tomar decisões*. Isso pode indicar que a lógica deveria estar no `MusicContext`. • Um método que chama vários getters de um objeto para calcular algo.	• **Mover Método (Move Method):** Leve o método para a classe que contém os dados que ele mais usa. Por exemplo, `aumentarOitava()` poderia ser um método do próprio `MusicContext`.
6. Obsessão por Tipos Primitivos (Primitive Obsession) Ugo excessivo de tipos básicos em vez de objetos pequenos.	• Representar uma "nota musical" como uma `String` ("LA", "DO"). • Representar "volume" e "oitava" como `int` soltos. • Usar `Map` ou `List` para estruturas de dados complexas.	• **Substituir Primitivo por Objeto (Replace Primitive with Object):** Crie classes `Nota`, `Volume`, `Oitava` para encapsular dados e comportamentos relacionados (ex: a nota saber seu valor MIDI).
7. Switch Statements (ou if-else longos) Longas cadeias de condicionais que verificam o mesmo tipo.	• Um `switch` gigante ou sequência de `if-else` baseada no caractere de entrada para decidir a ação musical. Isso é uma forte indicação de que você precisa de polimorfismo.	• **Substituir Condicional por Polimorfismo (Replace Conditional with Polymorphism):** Crie uma interface `RegraMapeamento` e implementações concretas para cada tipo de caractere ou regra (`RegraNota`, `RegraInstrumento`, `RegraVolume`).
8. Cirurgia com Espingarda (Shotgun Surgery) Uma única	• Ao adicionar um *novo* tipo de efeito musical (ex: vibrato), você precisa modificar a `RuleEngine`, o `MusicContext` e o `MidiGenerator` simultaneamente.	• **Mover Função (Move Function)** e **Mover Campo (Move Field):** Concentre as responsabilidades relacionadas em uma única classe. • **Combinar Funções em Classe (Combine**

Smell (Mau-Cheiro)	Onde Observar no Projeto (Sinais e Sintomas)	Como Corrigir (Refatorações Sugeridas)
mudança lógica obriga a modificar várias classes diferentes.		**Functions into Class):** Isole a lógica do novo efeito em sua própria classe para que futuras mudanças sejam locais.
9. Comentários (Comments) Não que comentários sejam ruins, mas geralmente são usados para disfarçar código ruim.	• Comentários que explicam "o que" o código faz (o código deveria ser autoexplicativo). • Blocos de código comentado que "podem ser úteis um dia". • Javadocs que repetem a assinatura do método.	• **Extrair Método (Extract Method):** Se um comentário explica um bloco de código, transforme o bloco em um método com o nome do comentário. • **Renomear Variável (Rename Variable):** Use nomes claros para que os comentários se tornem desnecessários. • **Deletar:** Apague código comentado. O Git guarda o histórico.
10. Classes Alternativas com Interfaces Diferentes Classes que fazem a mesma coisa, mas têm métodos com nomes diferentes.	• Você tem uma classe `MidiPlayer` com um método `tocarNotaMidi()` e uma classe `AudioPlayer` com um método `reproduzirSom()`. Ambos fazem a mesma coisa, mas não podem ser usados de forma intercambiável.	• **Extrair Interface (Extract Interface):** Crie uma interface `Player` com um método `reproduzir()` e faça ambas as classes a implementarem. Isso permite usar polimorfismo e desacoplar o resto do sistema da implementação concreta.

Como Usar Este Checklist

1. **Autoavaliação:** Cada grupo autor cujo trabalho será avaliado (revisado) pode usar esta lista para verificar seu próprio código antes de compartilhá-lo com o grupo revisor.

2. **Revisão por Pares:** Após ter acesso ao *branch* principal do repositório, o grupo avaliador deve se reunir e revisar o código usando este checklist.
3. **Avaliação:** Ao avaliar, cada membro do grupo avaliador pode usar a lista para verificar o código e criar a sua lista de bad smells. Depois o grupo avaliador se reúne para compilar a lista de cada membro e decidir UMA LISTA FINAL a ser entregue pelo grupo avaliador.
4. **Identificação de Pontos de Refatoração:** Mantenham uma lista dos smells encontrados. Isso ajudará a planejar as sessões de refatoração do grupo cujo código foi avaliado e a demonstrar ao professor como a qualidade do código está evoluindo.

Observação: Lembre-se: o objetivo não é um código 100% "perfeito" e imutável, mas sim um código que seja **flexível, compreensível e preparado para crescer**. Este checklist é uma ferramenta para guiar essa jornada.

Bom trabalho!!